Lecture # 08
Pipelining Architecture

PARALLEL AND DISTRIBUTED COMPUTING

CS-482

Introduction to Pipelining

- **Definition**: Pipelining is a technique in computer architecture where multiple instruction stages are overlapped in execution to improve performance.
- **Need for Pipelining**:
  - Enhances CPU efficiency
  - Increases instruction throughput
  - Reduces execution time

What is Pipelining?

Pipelining is a technique used in computer architecture where multiple instruction stages are executed in parallel, improving CPU performance. It divides the execution process into several stages, with each stage handling a different instruction simultaneously. This increases instruction throughput and allows the CPU to process multiple instructions at different stages in a single clock cycle.

Need for Pipelining

Pipelining is essential in modern processors for several reasons:

1. **Improved Performance & Speed**
   - Without pipelining, the CPU executes one instruction at a time, leading to underutilization of processing units.
   - With pipelining, multiple instructions are processed simultaneously, increasing instruction throughput.

2. **Efficient Resource Utilization**
   - Pipelining ensures that different functional units (fetch, decode, execute, etc.) work in parallel, reducing idle time.

3. **Higher CPU Clock Speeds**
   - Since each instruction is broken into smaller tasks, they can be executed faster, allowing for higher clock frequencies.

4. **Minimizing Instruction Execution Delay**
   - Without pipelining, each instruction must complete before the next begins. Pipelining overlaps execution, reducing overall execution time.

5. **Scalability for Complex Architectures**
   - Advanced architectures like RISC (Reduced Instruction Set Computing) heavily depend on pipelining for optimal performance.

6. **Better Utilization of Multi-Core and Superscalar Architectures**
   - Modern CPUs use multiple execution units. Pipelining helps manage multiple instruction flows efficiently.

Types of Pipelining

- **Hardware Pipelining**: Uses physical components to implement multiple execution stages.
- **Software Pipelining**: Compiler optimizations to rearrange instructions for better performance.

1. Hardware Pipelining (Processor-Level Pipelining)

Hardware pipelining refers to the physical design of a processor where different hardware units are dedicated to specific pipeline stages. It improves execution efficiency by enabling parallel processing of instructions.

Types of Hardware Pipelining:

1. **Instruction Pipelining**
   - Used in CPUs to execute multiple instructions simultaneously by breaking them into different stages (Fetch, Decode, Execute, Memory Access, Write Back).
   - Example: The 5-stage pipeline in RISC processors.

2. **Arithmetic Pipelining**
   - Used in ALUs (Arithmetic Logic Units) for executing complex mathematical operations like floating-point addition and multiplication.
   - Example: Floating-Point Unit (FPU) pipelining in modern processors.

3. **Data Pipelining**
   - Used in memory systems to transfer data efficiently between components (cache, main memory, I/O devices).
   - Example: DMA (Direct Memory Access) controllers use data pipelining.

2. Software Pipelining

Software pipelining is a compiler optimization technique where loops are rearranged to allow parallel execution of instructions. It enhances performance without requiring additional hardware modifications.

Types of Software Pipelining:

1. **Loop Unrolling**
   - Increases instruction-level parallelism by reducing loop control overhead.
   - Example: Instead of running a loop 10 times, it executes 2 iterations per cycle (5 cycles in total).
2. **VLIW (Very Long Instruction Word) Pipelining**
   - Compilers schedule multiple operations into a single instruction word, executed in parallel.
   - Example: Used in DSP (Digital Signal Processing) processors.
3. **Speculative Execution**
   - Instructions are executed before their conditions are confirmed, reducing pipeline stalls.
   - Example: Used in branch prediction to reduce control hazards.
4. **Software Parallelism in Multi-Core Processors**
   - Breaking tasks into multiple threads to utilize multiple cores effectively.
   - Example: OpenMP and CUDA-based parallel programming.

Tradeoff Between Cost and Performance in Pipelining

Pipelining improves system performance by increasing instruction throughput, but it also introduces higher costs due to hardware complexity, power consumption, and design challenges. The tradeoff between cost and performance must be carefully balanced depending on the application and system requirements.

Tradeoff Between Cost and Performance in Pipelining

Scenario	High Performance	Low Cost
Simple CPU Design (e.g., embedded systems)	Few pipeline stages, lower complexity	Reduced hardware cost, lower power usage
High-Speed Processors (e.g., gaming, AI, HPC)	Deep pipelining, out-of-order execution	Expensive fabrication and higher power
Superpipelining (More stages per instruction)	Increases clock speed and performance	Increases hazard handling complexity
Superscalar (Multiple pipelines)	Can execute multiple instructions per cycle	Requires more execution units, raising cost
Software Pipelining Optimization	Compiler-based optimization for performance	No extra hardware cost but adds complexity

Optimizing the Tradeoff

To balance cost and performance:

- **Select the right pipeline depth** – Avoid excessive stages that increase hazards.
- **Use hazard mitigation techniques** – Register forwarding and branch prediction improve efficiency.
- **Optimize instruction scheduling** – Software pipelining can reduce hardware complexity.
- **Choose the right architecture** – RISC favors deeper pipelines, while CISC may avoid excessive pipelining.
- For cost-sensitive applications (e.g., embedded systems, IoT), simpler pipelines with fewer stages are preferable.
- For performance-critical applications (e.g., gaming, AI, supercomputing), deeper pipelines with advanced hazard handling are necessary, despite higher costs.
- The ideal balance depends on system requirements, workload, and power constraints.

Stages in Pipelining

1. **Instruction Fetch (IF)**: Fetch the instruction from memory.
2. **Instruction Decode (ID)**: Decode the instruction and determine operands.
3. **Execute (EX)**: Perform the operation.
4. **Memory Access (MEM)**: Read/write data from/to memory.
5. **Write Back (WB)**: Store the result in a register.

1. Instruction Fetch (IF)

- Retrieves the next instruction from memory (RAM or cache).
- Updates the **Program Counter (PC)** to point to the next instruction.
- **Possible Issue**: Branch instructions may cause incorrect instruction fetching (Control Hazard).

Optimization Techniques:

- **Instruction Prefetching**: Fetches multiple instructions in advance.
- **Branch Prediction**: Predicts the outcome of branch instructions to avoid stalls.

2. Instruction Decode (ID)

- Decodes the fetched instruction to determine the operation type (arithmetic, load/store, branch, etc.).
- Identifies source registers and operands.
- Sends control signals to the execution unit.

Optimization Techniques:

- **Register Renaming**: Resolves data dependency issues.
- **Out-of-Order Execution**: Allows instructions to execute in a different order for efficiency.

3. Execute (EX)

- Performs the actual computation or operation using the **ALU (Arithmetic Logic Unit)**.
- If it’s an arithmetic instruction, the ALU performs the calculation.
- If it’s a branch instruction, branch target addresses are calculated.

Optimization Techniques:

- **Operand Forwarding**: Bypasses unnecessary waiting by sending results directly to the next stage.
- **Multiple Execution Units**: Allows parallel processing of instructions.

4. Memory Access (MEM)

- Accesses memory for load/store instructions.
- **Load Instruction (LW)**: Reads data from memory into a register.
- **Store Instruction (SW)**: Writes register data into memory.

Optimization Techniques:

- **Cache Memory**: Reduces memory access delays.
- **Memory Hierarchy Optimization**: Uses multi-level caching (L1, L2, L3).

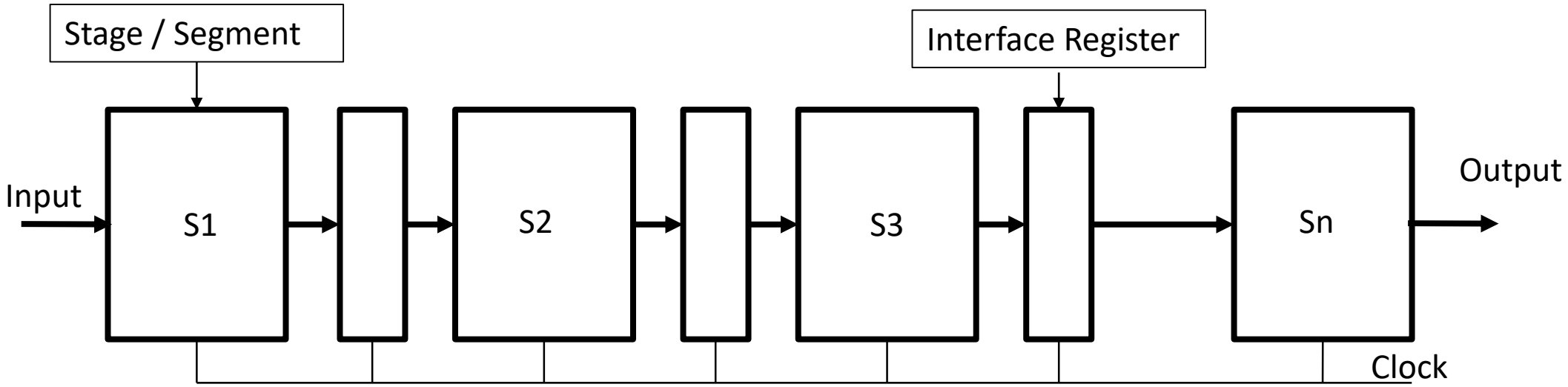
5. Write Back (WB)

- Stores the result of execution (ALU result or memory read) into the destination register.
- Ensures data is available for future instructions.

Optimization Techniques:

- **Register Forwarding**: Reduces waiting by immediately using updated register values.
- **Write Buffers**: Improves memory write performance.

Graphical Representation of Pipelining



Example of Instruction Execution in Pipelining

Assume we have three instructions:

1. ADD R1, R2, R3

2. SUB R4, R5, R6

3. LOAD R7, 0(R8)

In a pipelined processor, execution happens as follows:

Cycle	IF (Fetch)	ID (Decode)	EX (Execute)	MEM (Memory)	WB (Write Back)
1	ADD				
2	SUB	ADD			
3	LOAD	SUB	ADD		
4	Next Inst	LOAD	SUB	ADD	
5	...	Next Inst	LOAD	SUB	ADD
6	...	...	Next Inst	LOAD	SUB

Example of Instruction Execution in Pipelining

Instead of executing one instruction at a time, multiple instructions are overlapping in execution.

This improves throughput significantly compared to non-pipelined execution.

The five-stage pipeline (IF, ID, EX, MEM, WB) is widely used in **RISC architectures** like MIPS. More complex architectures (e.g., **Intel, AMD CPUs**) may have deeper pipelines (10+ stages) for higher performance, but they must handle **hazards and stalls** efficiently.

Pipeline Execution Diagram(space time Diagram)

Clock Cycle →	1	2	3	4	5	6	7	8	9
Instr 1	IF	ID	EX	MEM	WB				
Instr 2		IF	ID	EX	MEM	WB			
Instr 3			IF	ID	EX	MEM	WB		
Instr 4				IF	ID	EX	MEM	WB	
Instr 5					IF	ID	EX	MEM	WB

Serial Execution (Without Pipelining)

Clock Cycle →	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Instr 1	IF	ID	EX	MEM	WB										
Instr 2					IF	ID	EX	MEM	WB						
Instr 3							IF	ID	EX	MEM	WB				
Instr 4									IF	ID	EX	MEM	WB		
Instr 5											IF	ID	EX	MEM	WB

Pipelined vs Non-Pipelined Execution

Total Cycles Required for 15 Instructions in Serial Execution: 75 Cycles
(Each instruction takes 5 cycles, executed sequentially)

Total Cycles for 15 Instructions in Pipelined Execution: 19 Cycles
(Only the first instruction takes 5 cycles, then a new instruction enters every cycle)

Pipelined vs Non-Pipelined Execution

Feature	Pipelined Execution	Non-Pipelined Execution
Instruction Execution	Overlapped	Sequential
Speedup	High	Low
Efficiency	High	Low
Utilization	Better resource utilization	Poor resource utilization

Characteristics of Pipelining

1. **Multiple Instructions Executed Simultaneously:** Pipelining allows multiple instructions to be in different execution stages at the same time, improving CPU throughput.
2. **Segmentation of the Instruction Cycle:** The instruction cycle is divided into distinct stages (e.g., Fetch, Decode, Execute, Memory, Write Back). Each stage performs a specific task.
3. **Overlapping of Operations:** Different instructions are processed in different pipeline stages simultaneously, leading to efficient utilization of CPU resources.
4. **Increased Throughput:** The number of instructions completed per unit time is higher compared to non-pipelined execution.
5. **Reduced Execution Time:** Once the pipeline is full, an instruction is completed in nearly every cycle, reducing the average execution time per instruction.

Characteristics of Pipelining

1. **Pipeline Depth (Stages Count):** The number of stages in a pipeline determines the depth of pipelining. More stages generally improve throughput but may increase complexity.
2. **Pipeline Hazards:** Pipelining can face **structural hazards** (resource conflicts), **data hazards** (data dependencies), and **control hazards** (branch instructions).
3. **Speedup Factor:** Ideally, a pipeline can achieve a speedup equal to the number of stages, but practical speedup is limited due to stalls and hazards.
4. **Dependency Management:** Modern processors include techniques like forwarding and branch prediction to handle dependencies and improve pipeline efficiency.
5. **Latency vs. Throughput Trade-off:** While pipelining improves instruction throughput, it does not reduce the execution time of a single instruction (latency remains the same).

Example: 8 Instructions in a 5-Stage Pipeline

We consider a **5-stage RISC pipeline** with the following stages:

1. **IF (Instruction Fetch)** – Fetch instruction from memory.
2. **ID (Instruction Decode)** – Decode instruction & read registers.
3. **EX (Execute)** – Perform ALU operation.
4. **MEM (Memory Access)** – Read/write memory (for load/store).
5. **WB (Write Back)** – Write results to registers.

Instructions in the Program

1. LOAD R1, 0(R2) – Load value from memory to R1
2. ADD R3, R1, R4 – Add R1 and R4, store in R3
3. SUB R5, R3, R6 – Subtract R6 from R3, store in R5
4. MUL R7, R5, R8 – Multiply R5 and R8, store in R7
5. AND R9, R7, R10 – Bitwise AND on R7 and R10, store in R9
6. OR R11, R9, R12 – Bitwise OR on R9 and R12, store in R11
7. XOR R16, R11, R14 – XOR operation, store in R16
8. STORE R16, 0(R15) – Store R16 into memory

Pipeline Execution Table (Clock Cycles)

Clock Cycle →	1	2	3	4	5	6	7	8	9	10	11	12
LOAD R1	IF	ID	EX	MEM	WB							
ADD R3		IF	ID	EX	MEM	WB						
SUB R5			IF	ID	EX	MEM	WB					
MUL R7				IF	ID	EX	MEM	WB				
AND R9					IF	ID	EX	MEM	WB			
OR R11						IF	ID	EX	MEM	WB		
XOR R16							IF	ID	EX	MEM	WB	
STORE R16								IF	ID	EX	MEM	WB

Observations

Instruction Overlap: Each instruction starts before the previous one finishes, improving efficiency.

Speedup Factor: Instead of taking 8 × 5 = 40 cycles in a serial execution, pipelining completes all 8 instructions in 12 cycles.

Hazards & Dependencies: If dependencies exist (e.g., ADD R3, R1, R4 depends on LOAD R1), stalls or forwarding techniques are needed to avoid incorrect results.

Space-Time Diagram

A **Space-Time Diagram** represents how instructions progress through the pipeline **over clock cycles**.

Clock Cycle →	1	2	3	4	5	6	7	8	9	10	11	12
LOAD R1	IF	ID	EX	MEM	WB							
ADD R3		IF	ID	EX	MEM	WB						
SUB R5			IF	ID	EX	MEM	WB					
MUL R7				IF	ID	EX	MEM	WB				
AND R9					IF	ID	EX	MEM	WB			
OR R11						IF	ID	EX	MEM	WB		
XOR R16							IF	ID	EX	MEM	WB	
STORE R16								IF	ID	EX	MEM	WB

Speedup Calculation

For **N instructions** and **k stages**,

- $\boldsymbol{Non - Pipelined\ Execution\ Time} = N \times k\ cycles$
- $\boldsymbol{Pipelined\ Execution\ Time} = k + (N - 1)\ cycles$
- $Speedup = \frac{Time\ taken\ without\ pipelining}{Time\ taken\ with\ pipelining}$

For **8 instructions in a 5-stage pipeline**:

- $\boldsymbol{Non - Pipelined\ Time} = 8 \times 5 = 40 cycles$
- $\boldsymbol{Pipelined\ Time} = 5 + (8 - 1) = 12\ cycles$
- $\boldsymbol{Speedup} = \frac{40}{12} \approx 3.33$

Clock Cycles Per Instruction (CPI)

The **Clock Cycles Per Instruction (CPI)** in a pipelined processor is calculated as:

$$CPI = \frac{Total\ CLock\ Cycles}{Total\ Instructions}$$

For a Pipelined Processor:

The total clock cycles for executing **N** instructions in a pipeline with **K** stages is:

$$Clock\ Cycles = K + (N - 1)$$

Thus, the CPI formula for a pipelined processor becomes:

$$CPI_{pipeline} = \frac{K+(N-1)}{N}$$

For large **N**, this approaches **1**, meaning an ideal pipeline can achieve a CPI close to **1**, indicating high efficiency.

For a Non-Pipelined Processor:

Each instruction takes **K** cycles, so:

$$Clock\ Cycles = K + N$$

Thus, the CPI for a non-pipelined processor is:

$$CPI_{non-pipeline} = \frac{K*N}{N} = K$$

Comparison:

- **Pipelined CPU**: $CPI \approx 1$ (for large **N**)
- **Non-Pipelined CPU**: $CPI = K$

Example

Given:

- $\boldsymbol{Number\ of\ instructions}\ N = 1000$
- $\boldsymbol{Number\ of\ pipeline\ stages}\ K\ (let's\ assume\ \boldsymbol{K\ =\ 5}\ as\ a\ common\ case)$

The formula for total clock cycles in a pipelined processor:

$$Clock\ Cycles = K + (N - 1)$$

Substituting values:

$$Clock\ Cycles = 5 + (1000 - 1) = 1004$$

The **CPI (Clock Cycles Per Instruction)** is:

$$CPI = \frac{Total\ CLock\ Cycles}{Total\ Instructions}$$

$$CPI = \frac{1004}{1000} = 1.004 \approx 1$$

Interpretation:

- The CPI is **very close to 1**, which indicates **high efficiency** of pipelining.
- As **N** increases, CPI approaches **1** in an ideal pipeline.

Utilization Formula

Pipeline **utilization** measures how efficiently the pipeline stages are being used. It is given by:

$$Utilization = \frac{Busy\ Time\ of\ Pipeline}{Total\ Time\ Available}$$

In a pipelined architecture, the **total available time** is the product of **number of stages** (K) and **total clock cycles**:

$$Utilization = \frac{N}{K+(N-1)}$$

Where:

- $N = Number\ of\ instructions$
- $K = Number\ of\ pipeline\ stages$

Pipeline Execution Diagram(space time Diagram)

Clock Cycle →	1	2	3	4	5	6	7	8	9
Instr 1	IF	ID	EX	MEM	WB				
Instr 2		IF	ID	EX	MEM	WB			
Instr 3			IF	ID	EX	MEM	WB		
Instr 4				IF	ID	EX	MEM	WB	
Instr 5					IF	ID	EX	MEM	WB

Serial Execution (Without Pipelining)

Clock Cycle →	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
Instr 1	IF	ID	EX	MEM	WB																
Instr 2					IF	ID	EX	MEM	WB												
Instr 3									IF	ID	EX	MEM	WB								
Instr 4													IF	ID	EX	MEM	WB				
Instr 5																	IF	ID	EX	MEM	WB

Example Calculation:

Let's assume:

- $N = 1000\ instructions$
- $K = 5\ pipeline\ stages$

Using the formula:

$$Utilization = \frac{N}{K + (N - 1)}$$

$$Utilization = \frac{1000}{1000 + (5 - 1)}$$

Numerical

A **5-stage** pipeline system consists of the following processing delays at each stage:

- **Stage 1 (S1) Delay:** 180 ns
- **Stage 2 (S2) Delay:** 140 ns
- **Stage 3 (S3) Delay:** 170 ns
- **Stage 4 (S4) Delay:** 130 ns
- **Stage 5 (S5) Delay:** 160 ns

Registers are placed between each stage to store intermediate results, with each register introducing a delay of **8 ns**. The system operates with a constant clock cycle determined by the slowest stage plus the register delay.

Questions:

1. Determine the **clock cycle time** for this pipeline.
2. Calculate the **total time required** to process **1200 data items** using the pipeline.
3. Compare this with the **non-pipelined execution time**, assuming all stages execute sequentially for each instruction.
4. Compute the **speedup factor** of pipelining over non-pipelining.

Problem

A processor operates at a frequency of **3 GHz** and takes **5 clock cycles per instruction** on average. The system is upgraded to a **6-stage pipelined processor**, but due to internal pipeline overheads, the clock speed is reduced to **2.5 GHz**. Assume ideal pipelining with no stalls.

Question:

What is the speedup achieved with the pipelined processor?

Hazards in Pipelining

- **Data Hazards**: Dependency between instructions.
- **Structural Hazards**: Limited hardware resources.
- **Control Hazards**: Branch instructions affecting execution flow.

Data Hazards and Types

- **Read After Write (RAW)**: Instruction depends on previous instruction result.
- **Write After Read (WAR)**: Later instruction writes before an earlier instruction reads.
- **Write After Write (WAW)**: Two instructions write to the same register in the wrong order.

Structural Hazards

- Occurs when **hardware resources are limited** (e.g., single memory unit for instruction and data fetch).
- Solutions:
  - Increase hardware units
  - Instruction scheduling

Control Hazards

- Caused by **branch instructions** affecting the instruction flow.
- Solutions:
  - **Branch Prediction**
  - **Delay Slots**
  - **Speculative Execution**

Register Renaming for Data Hazard Resolution

Register Renaming is a technique used to eliminate data hazards (RAW, WAR, and WAW) in pipelining by assigning different physical registers to the same logical register.

This helps avoid conflicts between instructions trying to read or write to the same register.

Commonly used in modern processors that support out-of-order execution and superscalar architectures.

Formulas in Pipelining

- **Clock Cycles in Pipelining:** $T_{pipeline} = k + (n - 1)$ where k = Number of Stages, n = Number of Instructions
- **CPI (Clock Per Instruction):** $CPI = \frac{Total\ Cycles}{Total\ Instructions}$
- **Speedup Formula:** $S = \frac{Execution\ Time_{Non-Pipeline}}{Execution\ Time_{Pipeline}}$
- **Utilization Formula:** $Utilization = \frac{k}{k+(n-1)}$